Locally Checkable Problems in Distributed Computing

Dennis Olivetti

Last Update: June 4, 2026

Contents

Chapter 1

Introduction

The goal of this book is to provide an overview of recent results about locally checkable problems in the distributed setting. Locally checkable problems are problem for which, if we are given a solution, we can efficiently check its validity. We will mainly consider the LOCAL model of distributed computing, a synchronous message-passing model, where a graph represents a communication network, and neither the size of the messages nor the computational power of the machines is restricted. The considered complexity measure is the number of communication rounds required to solve a given problem. We will address the following topics:

- Distributed Complexity Theory.
  - What are the possible distributed time complexities of locally checkable problems?
  - Does randomness help? How much? Is shared randomness more powerful than private randomness? How about quantum?
  - Can we automate our work, that is, can we automatically determine the complexity of a given problem?
- Lower Bound Techniques.
  - We will mainly talk about the round elimination technique, which has been used to prove many lower bounds over the years.
  - We will discuss other techniques, such as Mark's technique, KMW, and indistinguishability arguments.
- Connections with other fields.
  - There are deep connections between the distributed setting and a mathematical field called *descriptive combinatorics*.
  - There are deep connetions bewtween the distributed setting and *finitary factors of iid processes*, a topic coming from the area of analysis of randomized processes.

Chapter 2

Preliminaries

2.1 The LOCAL model

We model a network as a graph $G = (V, E)$, where the nodes represent machines, and the edges represent communication links. Nodes may be assigned unique IDs. This model works as follows:

- Time proceeds in synchronous steps (machines share a clock).
- At the beginning, each node only knows its own input and its degree. The input of a node can be, e.g., its ID, the knowledge of the size $n = |V|$ of the graph, the maximum degree Δ, and/or some additional problem-specific inputs.
- Then, computation proceeds in rounds. At each round, each node can exchange messages with its neighbors, and update its own local state (each node has its own private memory, no memory is shared between nodes).
- Each node must eventually produce an output.

The complexity of an algorithm is the worst-case number of rounds that it requires to terminate. This complexity is often measured as a function of n and Δ.

In the LOCAL model, messages can be arbitrarily large, and computation is unbounded. There are many variants of the LOCAL model that one can consider, and they may have different power:

- Is n known by the nodes? Is a polynomial upper bound on n known?
- Do nodes have IDs?
- Do nodes have access to randomness? How much randomness? A finite or infinite bit-string? Do we require Las-Vegas or Monte-Carlo algorithms?
- Messages are arbitrarily large, but are they finite? Can we e.g. send real numbers?
- Can nodes locally perform uncomputable computation?

2.2 Locally Checkable Problems

Locally Checkable Problems (LCP) are problems for which it is easy to check whether a given solution is valid. In more detail, a locally checkable problem is a problem for which there exists a constant-time distributed algorithm called *verifier* satisfying that, given a solution:

- If the solution is correct, then the algorithm outputs *yes* on all nodes.
- If the solution is incorrect, then the algorithm outputs *no* on at least one node.

Many interesting problems are locally checkable, for example:

- Coloring problems. For example, consider the $(\Delta + 1)$-coloring problem, and assume that the verifier knows Δ. The verifier requires 1 round of communication, in which it checks that the color of the current node is in $\{1, \ldots, \Delta + 1\}$, and that it is different from the color of the neighbors.
- Maximal independent set. The verifier needs to check that if the current node is in, then all its neighbors must be out, and that if the current node is out, then at least one neighbor is in. Observe that the *Maximum* independent set problem is not locally checkable.
- Single source shortest paths (SSSP), or at least some variants of it. Consider for example the variant in which each node outputs its own distance and a pointer to the parent. Then the verifier needs to check that the distance of the parent, plus the weight of the edge connecting the node to the parent, is equal to the distance of the node, and that no other neighbor would give a strictly smaller distance.
- Sinkless orientation, that is, orienting the edges so that each node does not have all edges oriented incoming.
- List-coloring, where each node is given a list of colors as input, and each node needs to output a color from its own list, such that the resulting coloring is proper.

2.3 LCLs

In [12], a restricted class of LCP, called Locally Checkable Labelings (LCLs), was introduced. This class is restricted enough so that we can prove interesting results, but wide enough to still contain many problems of interest. In more detail, LCLs are LCPs with the following restrictions:

- The number of input and output labels is constant (i.e., it does not depend on n).
- It is assumed that $\Delta \in O(1)$.

An example of a problem that is an LCP but not an LCL is SSSP, because nodes need to output distances, and distances could depend on n.

The nice thing about this class of problems is that each problem has a finite description:

- Let r be the number of rounds taken by the verifier. Clearly, the output of the verifier can only depend on things at distance at most r from the node.
- Since Δ and r are in $O(1)$, and the number of labels is constant, there is a finite number of graphs that the verifier could see.
- Thus, we can list the ones that the verifier would accept. Hence, we can describe a problem by listing the accepted neighborhoods.

2.4 Distributed Complexity Theory

Being LCLs so restrictive, researchers managed to prove many interesting properties about them. Moreover, many techniques that have been initially developed to understand specific LCLs, have then been generalized to understand problems that fall outside of this class (e.g., round elimination). Common questions are the following:

- What are possible LCL complexities?
- Can we automatically decide the complexity of a given LCL?

- Does shared/private randomness help?
- Does quantum help?

2.4.1 Paths and Cycles

In the case of paths and cycles, LCLs have 3 possible complexities [1, 7, 11]:

- $O(1)$. Problems with this complexity often admit very easy algorithms. In the case of directed cycles with no inputs, we even know that all problems with this complexity can be solved in 0 rounds.
- $\Theta(\log^\star n)$. Problems with this complexity require breaking symmetry, but do not require global coordination. Examples are 3-coloring and MIS.
- $\Theta(n)$ and unsolvable problems. Problems with this complexity require global coordination. An example is 2-coloring, where, by fixing the output of a single node, we are forcing the output on all other nodes.

Surprisingly, LCLs on paths and cycles cannot have any other possible complexity, and randomization never helps. This fact has the following interesting implication: you can be sloppy. Suppose you prove that your favourite problem can be solved in $O(\log n)$ rounds via a simple randomized algorithm. You automatically get that the problem can also be solved in $O(\log^\star n)$ by a more clever algorithm, deterministically.

Another surprising fact is that everything is decidable, even for LCLs with inputs. In other words, we can feed the description of an LCL to a computer program, and the program is going to output the correct complexity, and a description of an optimal algorithm.

2.4.2 Trees

In the case of trees, there are the same possible complexities of paths and cycles, plus some more [10, 2, 8]:

- $\Theta(\log n)$ deterministic. These problems can either be $\Theta(\log n)$ randomized, or $\Theta(\log \log n)$ randomized.
- $\Theta(n^{1/k})$ for any $k \in \mathbb{N}^+$. For these problems, randomness does not help.

No other complexity is possible. Moreover, it is decidable whether a problem can be solved in $O(\log n)$ or not, and in the latter case, what the exact value of k is. Summarizing, some things are still decidable, and randomness can only help exponentially or not at all, and if it helps, it only helps for problems with deterministic complexity $\Theta(\log n)$. A major open question is whether it is decidable if a problem is $\Omega(\log n)$.

2.4.3 General graphs

In general graphs, things are entirely different. For deterministic complexities, the following is known [9, 5, 2]:

- There are problems with complexity $O(1)$.
- No problem with complexity in $\omega(1) - o(\log \log^\star n)$ exists.
- The region $\Omega(\log \log^\star n) - O(\log^\star n)$ is dense: informally, for any complexity in this range, we can find a problem with a complexity that is arbitrarily near to it.
- No problem with complexity in $\omega(\log^\star n) - o(\log n)$ exists.

- The region $\Omega(\log n)$ is dense.

In the case of general graphs, undecidability results are known. Very informally, they are proved by showing that it is possible to define problems that require outputting the execution of a Turing machine, and whether the problem is constant or not depends on the running time of the machine. About randomness, a surprising connection is known: if a problem has randomized complexity $o(\log n)$, then it has randomized complexity $O(T_{\mathrm{LLL}})$, where T_{LLL} is the distributed time complexity of the constructive Lovász Local Lemma [10]. It is known that T_{LLL} is in $\Omega(\log \log n)$ and in $O(\mathrm{poly} \log \log n)$, and a big open question is determining the exact complexity of LLL. Moreover, it is known that randomness can help in different regions. For example, it is known that, for all $k \in \mathbb{N}$, there are problems with deterministic complexity $\Theta(\log^{k+1} n)$ and randomized complexity $\Theta(\log^k n \log \log n)$ [3].

2.5 Easy vs Hard

A central question about LCLs is to determine whether a problem is *easy* or *hard*, which we define as follows.

- Easy problems are problems that can be solved in $O(\log^\star n)$. We call these problems easy because, in order to solve them, we do not need to exploit structural properties of the graph, and only some basic symmetry breaking is required. In some sense, these problems are truly local.
- If a problem cannot be solved in $O(\log^\star n)$, then we know it must have deterministic complexity $\Omega(\log n)$. If a problem falls in this latter case, then we say that the problem is hard. Problems of this type cannot be solved with local information: an algorithm running in $\Omega(\log n)$ rounds (for some large-enough constant hidden in the Ω notation) is guaranteed to see at least a node of degree ≤ 2 or a cycle. Algorithms of this type are not truly local; they need to see *something* in order to be able to solve the problem. Moreover, problems of this type have randomized complexity $\Omega(\log \log n)$. Observe that this is the sweet spot to obtain high-probability guarantees: either the algorithm sees *something* in the structure of the graph that it may be able to exploit, or it sees $\Omega(\log n)$ nodes. In the latter case, consider some randomized process that has constant success probability independently on each node. By seeing $\Omega(\log n)$ nodes, the algorithm will see a successful event with high probability, which it may be able to exploit.

2.6 Black-White formalism

An important technique used to prove lower bounds in the LOCAL model is the Round Elimination technique [6]. This technique does not directly work on LCLs, but requires the problem to be described with a specific language, called black-white formalism. It turns out that, at least on trees, this is not a restriction: we can convert any given LCL into a problem in the black-white formalism, and the two problems are equivalent, up to $O(1)$ additive rounds [4]. On general graphs, problems in the black-white formalism are a strict subclass of LCLs, but most natural problems can anyway be expressed in the black-white formalism.

Chapter 3

Distributed Complexity Theory

[TODO: summarize all papers about LCLs, with proof sketches]

Chapter 4

Connections with Other Fields

[TODO: descriptive combinatorics, ffiid]

Chapter 5

Black-White Formalism

The black-white formalism allows to define (some) LCLs in a concise way. While there are many variants that we will consider later, the most basic setting is the following:

- We assume that we are in a (Δ, δ)-biregular graph, where white nodes have degree Δ and black nodes have degree δ.
- There are no input labels.
- We need to output one label on each edge.
- We have a set (called *white constraint*) of *white allowed configurations*, which are multisets of size Δ that denote valid labelings of the edges incident to the white nodes.
- We have a set (*black constraint*) of *black allowed configurations*, which are multisets of size δ that denote valid labelings of the edges incident to the black nodes.

In other words, a problem Π in the black-white formalism is a triple (Σ, W, B), where Σ is the (finite) set of possible labels, W is a finite multiset, where each multiset has size Δ, and B is a multiset, where each multiset has size δ. Note that the definition of a problem Π is just this, in particular Σ is finite and there is no n involved: we cannot define problems where the number of labels depends the size of the graph.

5.1 Sinkless Orientation on Bipartite Graphs

Let's see a simple example (illustrated in Figure 5.1). Consider the following setting: we have the promise that our graph is 3-regular and 2-colored. We want to define a problem that requires to orient the edges of the graph such that no node is a sink, that is, each node must have at least one edge oriented outwards. We can define this problem in the black-white formalism as follows.

- $\Sigma = \{\mathsf{A}, \mathsf{B}\}$
- $W = \{\{\mathsf{A}, \mathsf{A}, \mathsf{A}\}, \{\mathsf{A}, \mathsf{A}, \mathsf{B}\}, \{\mathsf{A}, \mathsf{B}, \mathsf{B}\}\}$
- $B = \{\{\mathsf{B}, \mathsf{B}, \mathsf{B}\}, \{\mathsf{B}, \mathsf{B}, \mathsf{A}\}, \{\mathsf{B}, \mathsf{A}, \mathsf{A}\}\}$

The intuition is the following:

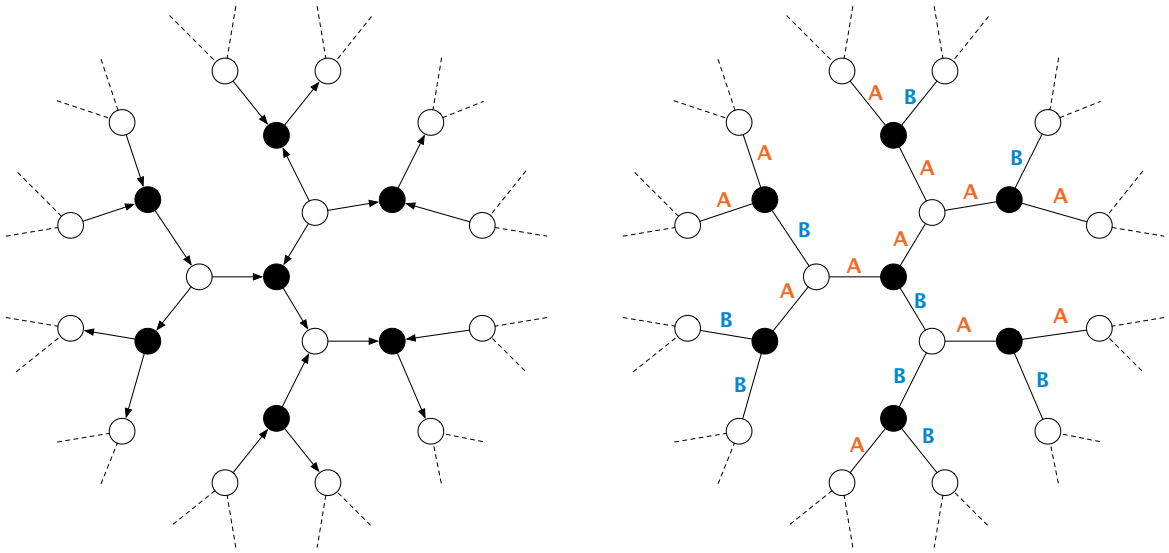
- Each edge gets exactly one label, A or B. If an edge is labeled A, then the edge is oriented from the white node to the black node, while if an edge is labeled B, then the edge is oriented from the black node to the white node.
- The white constraint W allows every configuration of 3 labels that contains at least one A, that is, at least one edge oriented away from the white node.

- The black constraint B allows every configuration of 3 labels that contains at least one B, that is, at least one edge oriented away from the black node.

In order to describe problems in the black-white formalism more compactly, we use the following notation. Instead of writing, e.g., $\{\mathsf{A}, \mathsf{A}, \mathsf{A}\}$, we simply write A A A, or even A^3. Hence, the white constraint becomes $\{\{\mathsf{A}^3\}, \{\mathsf{A}^2\ \mathsf{B}\}, \{\mathsf{A}\ \mathsf{B}^2\}\}$. To make the notation even more compact, we write these three allowed configurations as a single one: $\mathsf{A}\ [\mathsf{AB}]^2$. Think of it as a regular expression: we pick an A, and then we can pick A or B, twice. We call this type of configurations, *condensed configurations*. Sometimes we even omit the square brackets, and we just write $\mathsf{A}\ \mathsf{AB}^2$. This is the format that *round eliminator* (the tool that automatically applies the round elimination technique) expects. In order to describe a whole problem compactly, we omit Σ, we write a condensed configuration on each line, and we separate the white and the black constraint by an empty line. Hence, sinkless orientation is compactly defined as follows:

$\mathsf{A}\ \mathsf{AB}^2$

$\mathsf{B}\ \mathsf{AB}^2$



(a) A solution of bipartite sinkless orientation on bipartite 2-colored graphs.

(b) The same solution, encoded in the black-white formalism.

Figure 5.1: On the left, a solution of bipartite sinkless orientation as one would naturally imagine it. On the right, the same solution encoded in the black-white formalism. Each edge oriented from a white node to a black node gets the label A, while each edge oriented from a black node to a white node gets the label B.

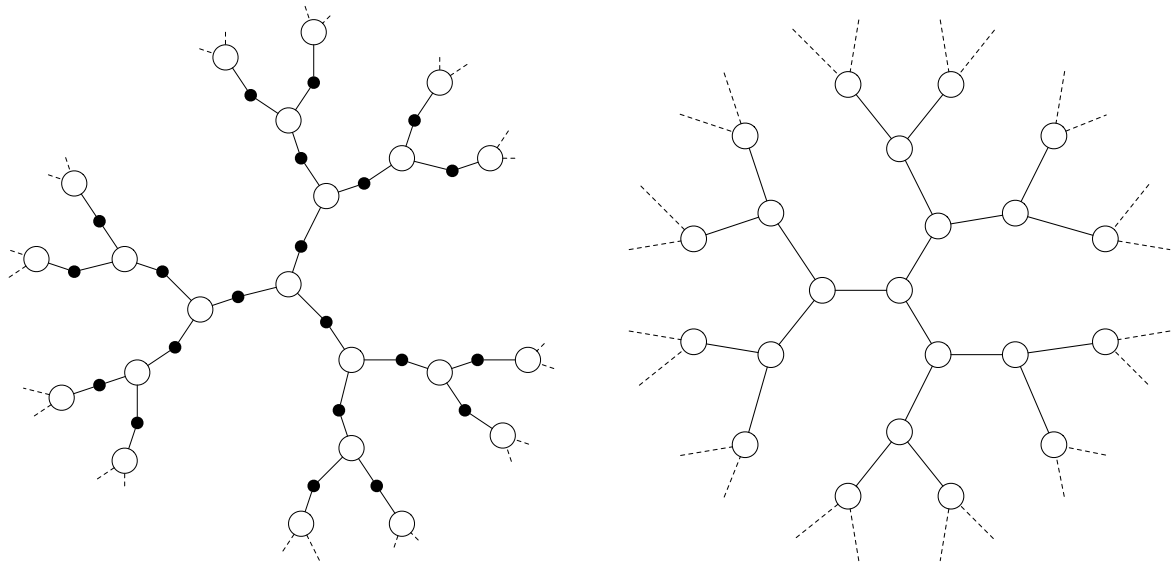
5.2 Node-Edge Formalism

The above example was in bipartite 2-colored graphs. However, the same formalism can be used to define problems in standard graphs. The idea is to just take $\delta = 2$ (see Figure 5.2). Observe that, in this case, each black node is in the middle of an edge connecting two white nodes. Let's just ignore these black nodes and pretend they do not exist. We get the following setting:

- We assume that we are in a Δ-regular graph.

- There are no input labels.
- We need to output one label on each *half-edge* (node-edge pair). In other words, we need to output two labels for each edge, one from each side.
- We have a list of *allowed node configurations*, which are multisets of size Δ that denote valid labelings of the edges incident to the nodes.
- We have a list of *allowed edge configurations*, which are multisets of size 2 that denote valid labelings of the edges.

This special case of the black-white formalism is called *node-edge formalism*.



(a) A $(3, 2)$-biregular graph. (b) A 3-regular graph.

Figure 5.2: By considering the graph on the left, and ignoring black nodes, we obtain the graph on the right. Observe that a labeling of the edges of the graph on the left corresponds to a labeling of the half-edges of the graph on the right.

5.3 Sinkless Orientation

Let's see an example of a problem encoded in the node-edge formalism (see Figure 5.3). We can define sinkless orientation as follows:

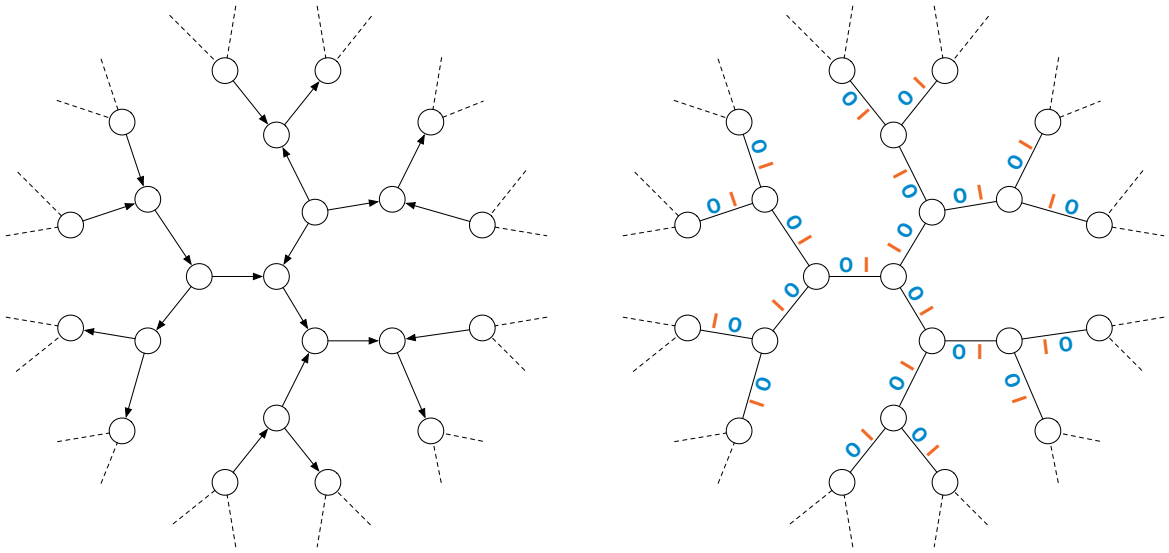
$\mathsf{O}\ \mathsf{IO}^2$

$\mathsf{I}\ \mathsf{O}$

In other words:

- The labels are I (oriented incoming) and O (oriented outgoing).
- Each node must have at least one O, that is, one edge oriented outgoing. But also two or three outgoing edges are fine, so all choices over the regular expression $\mathsf{O}\ \mathsf{IO}^2$, that is, $\mathsf{O}\ \mathsf{I}\ \mathsf{I}$, $\mathsf{O}\ \mathsf{O}\ \mathsf{I}$, $\mathsf{O}\ \mathsf{I}\ \mathsf{O}$, $\mathsf{O}\ \mathsf{O}\ \mathsf{O}$ are allowed. Note that $\mathsf{O}\ \mathsf{O}\ I$ and $\mathsf{O}\ \mathsf{I}\ \mathsf{O}$ are the same multiset, because the order does not matter.

- Edges must be properly oriented: if a node labels an edge I, then the other endpoint of the edge must label the edge O, and vice versa. We do not write O I in the list of allowed configurations because it is the same multiset as I O, and the order does not matter.



(a) A solution of sinkless orientation.

(b) The same solution, encoded in the node-edge formalism.

Figure 5.3: On the left, a solution of sinkless orientation as one would naturally imagine it. On the right, the same solution encoded in the node-edge formalism. Each edge gets the label O on the side oriented outgoing and O on the side oriented incoming.

5.4 Coloring

Let's see another example, 3-coloring 3-regular graphs (see Figure 5.4). We can define it in the node-edge formalism as follows.

R^3

G^3

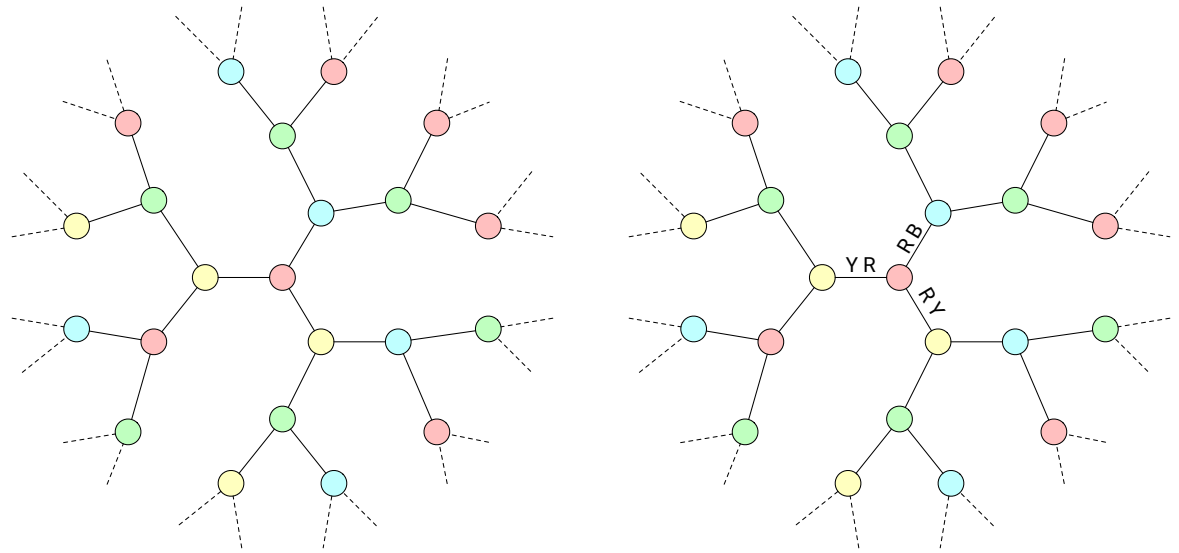
B^3

R GB

G B

In other words:

- Nodes can be colored red, green, or blue.
- Nodes colored, e.g., red, must output R on each of the three incident half-edges.
- On the edges, any configuration not consisting of the same color twice is allowed, that is, R G, R B, and G B are allowed, but, e.g., R R is not.



(a) A solution of 3-coloring in 3-regular graphs.

(b) An example of encoding in the node-edge formalism. The central node would, e.g., output R R R.

Figure 5.4: A solution of 3-coloring in 3-regular graphs, as one would naturally imagine it, and how it is encoded in the node-edge formalism.

5.5 Maximal Independent Set

Let's now consider a problem that is slightly harder to encode, Maximal Independent Set (MIS). In this problem, we need to select a subset of nodes that satisfy independence and maximality. While it may feel natural to try to encode it by using two labels (I and O, which stand for "in the set" and "out, not in the set"), this is not possible. In more detail, we could try to require nodes in the set to output I on each incident half-edge, and nodes not in the set to output O on each incident half-edge. Note that this would not work:

- There are valid solutions in which two nodes not in the set are neighbors, and hence we would need to allow O O on the edges.
- Hence, the node constraint contains the configuration O ... O, while the edge constraint contains the configuration O O.
- This implies that the obtained problem can be solved in 0 rounds of communication, by outputting O everywhere.
- The obtained problem is not *maximal independent set*, but just *independent set*!

In other words, we need one more label to ensure maximality. We can define MIS in the node-edge formalism, on 3-regular graphs, as follows.

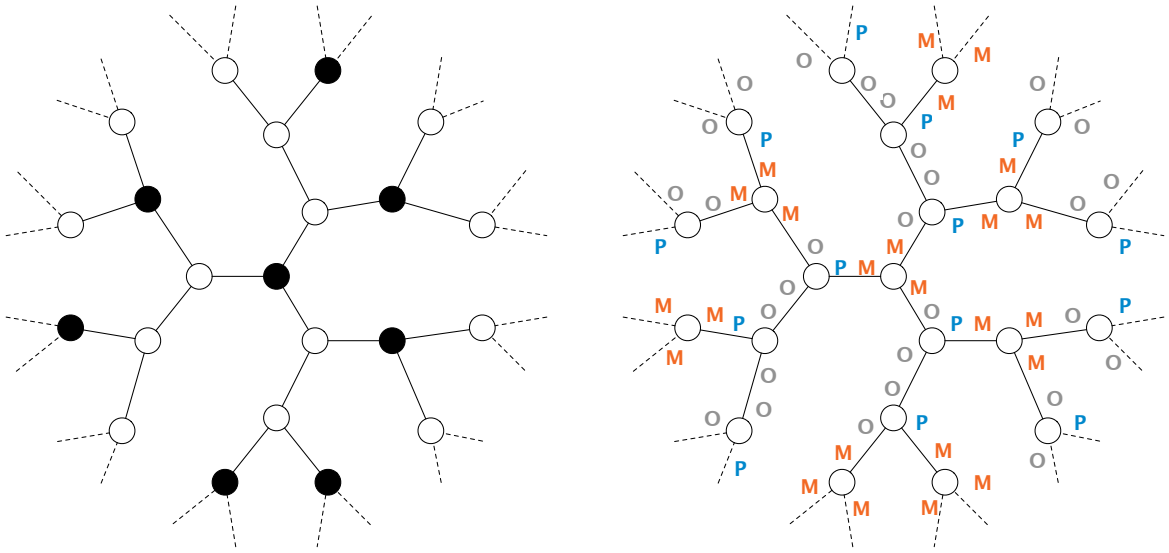
M^3
$\mathsf{P}\ \mathsf{O}^2$

$\mathsf{M}\ \mathsf{OP}$
$\mathsf{O}\ \mathsf{O}$

That is, MIS is encoded as follows:

- Nodes in the set output M^3.
- Nodes not in the set need to *prove* that they have at least one neighbor in the set, by outputting P towards them. On the other two edges, they output O.
- On the edges, we can see that P is only compatible with M (that is, the configuration $\mathsf{M}\ \mathsf{P}$ is the only one containing P), this ensures maximality. Then, nodes not in the set could have more than one neighbor in the set, and hence we allow $\mathsf{M}\ \mathsf{O}$. Moreover, nodes not in the set could be neighbors, and hence we allow $\mathsf{O}\ \mathsf{O}$.

See Figure 5.5 for an example.



(a) A solution of MIS.

(b) The same solution, encoded in the node-edge formalism.

Figure 5.5: On the left, a solution of MIS in a 3-regular graph, as one would naturally imagine it. On the right, the same solution encoded in the node-edge formalism. Each node in the set outputs M^3, while each node not in the set outputs P towards a neighbor that is in the set, and O towards the other two neighbors.

Let's call the usual MIS problem standard-MIS, and the version encoded in the node-edge formalism encoded-MIS. It is important to notice that standard-MIS and encoded-MIS are not exactly the same problem:

- Given a solution to encoded-MIS, we can immediately infer a solution to standard-MIS.
- However, given a solution to standard-MIS, it takes 1 round of communication to obtain a solution to encoded-MIS, because nodes not in the set would not otherwise know where to output P.

We need to keep this in mind when proving lower bounds: if we prove that encoded-MIS requires at least T rounds to be solved, we only obtain a lower bound of $T - 1$ rounds for standard-MIS. While in the case of MIS it does not matter (asymptotically), for some other problems it could matter, so we have to keep this in mind.

5.6 Bipartite Maximal Matching

Let's see another example in the black-white formalism, by considering the problem of computing a maximal matching on bipartite 2-colored graphs (BMM). Similarly as in the case of MIS, one

may be tempted to encode BMM by using two labels, but this is not possible, we need at least three. We can encode BMM, on $(3, 3)$-biregular graphs, as follows:

$\mathsf{M}\ \mathsf{O}^2$
P^3

$\mathsf{M}\ \mathsf{OP}^2$
O^3

See Figure 5.6 for an example. The intuition is the following:

- Edges that are part of the matching are labeled M.
- Then, if a white node is not matched, it needs to prove that all its neighbors are matched, so it points to all of them, by outputting P on all its incident edges.
- There are two types of black nodes, matched and unmatched. If a node is matched then it has exactly one edge labeled M, and then it can *accept* pointers. So all its other edges could be P or O. If a black node is not matched, then it needs to prove that all its white neighbors are matched. We do not need a new type of pointer for this, we can use O, because all non-matched edges incident to white matched nodes are labeled O.

Note that this is not the only possible way to encode BMM. One could use a more "symmetric" and intuitive version, as follows:

$\mathsf{M}\ \mathsf{PO}^2$
P^3

$\mathsf{M}\ \mathsf{PO}^2$
O^3

The intuition is the following:

- Matched edges get label M, then the other edges are either *white pointers* P or *black pointers* O.
- If a node is matched, then it outputs exactly one M, and then on the other edges everything else is allowed.
- If a white node is not matched, it outputs white pointers everywhere, that is, P^3.
- If a black node is not matched, it outputs black pointers everywhere, that is, O^3.
- Observe that if a white node is not matched, then all its edges are labeled P, and all black configurations containing P also contain M. Hence, all black neighbors of the white node are matched. The case of unmatched black nodes is symmetric.

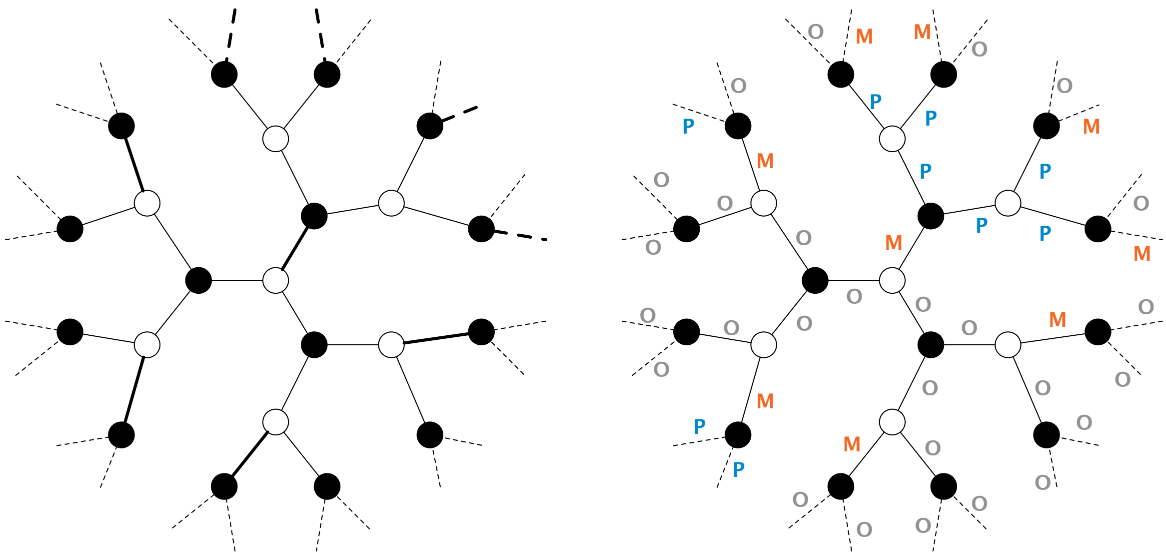
It turns out that the two variants that we have seen are equivalent, and it is now time to discuss an important detail: who outputs the labels? We will use the black-white formalism to prove upper and lower bounds via round elimination. In such a setting, it will be convenient to consider the following computational model:

- White nodes output labels on their incident edges.
- Black nodes participate in the computation, but they do not output anything.

- Hence, it is the job of the white nodes to output labels that satisfy, at the same time, white and black constraints on all nodes.

Now, a solution for the first variant *is also* a solution for the second variant. A solution for the second variant *can be converted*, without communication, into a solution for the first variant, if we consider the computational model just discussed: matched white nodes just need to change all their P into O. It is easy to see that the obtained solution is valid.

As a side note, if we enter the second variant of BMM into round eliminator, it automatically transforms the given problem into the first variant, because it sees the P in the condensed configuration M PO^2 of the white constraint as useless.



(a) A solution of BMM.

(b) The same solution, encoded in the black-white formalism.

Figure 5.6: On the left, a solution of BMM in a 3-regular graph, as one would naturally imagine it. On the right, the same solution encoded in the node-edge formalism. Edges in the matching get the label M, then we need to use P and O to ensure maximality.

5.7 Ruling Sets

Ruling sets are a family of problems parameterized by two positive integers α and β. An (α, β)-ruling set is a set of nodes that satisfies two properties:

- Nodes in the set are at distance at least α from each other.
- Each node not in the set has a node in the set at distance at most β.

Observe that the problem of computing a $(2,1)$-ruling set is the same as MIS. We will now see how to encode $(2,2)$-ruling sets on 3-regular graphs, in the node-edge formalism.

M^3

$\mathsf{P}\ \mathsf{O}^2$

$\mathsf{Q}\ \mathsf{V}^2$

$\mathsf{M}\ \mathsf{PO}$

$\mathsf{O}\ \mathsf{Q}$

OV^2

In other words:

- Nodes in the set output M^3, and M^2 is not in the edge constraint, to satisfy independence.
- Nodes not in the set need to output *pointers* to prove maximality. These pointers can be P or Q. However, P is only compatible with M, and hence nodes outputting $\mathsf{P}\ \mathsf{O}^2$ must be at distance 1 from at least one node in the set. Nodes at distance 2 from nodes in the set output $\mathsf{Q}\ \mathsf{V}^2$, and Q must point to an O, and hence to a node at distance 1, ensuring that all nodes outputting $\mathsf{Q}\ \mathsf{V}^2$ are at distance 2 from nodes in the set.
- Except for needing to output pointers, there are no further constraints, so on the edges we allow any choice over OV^2.

This is not the only way to encode $(2,2)$-ruling sets. We could have for example allowed $\mathsf{M}\ \mathsf{QV}$ on the edges, which would allow nodes outputting $\mathsf{Q}\ \mathsf{V}^2$ to be a distance 1 from nodes in the set. With both encodings, it would take 2 rounds to convert a solution for (standard) $(2,2)$-ruling sets into the version encoded in the node-edge formalism.

If we were to generalize this idea to $(2,\beta)$-ruling sets, for larger β, we would essentially do it as follows:

- We require nodes in the set to output M^3.
- For each $1 \le i \le \beta$, we would introduce two new labels P_i and O_i.
- Then, nodes at distance i from the set need to output $\mathsf{P}_i\ \mathsf{O}_i\ \mathsf{O}_i$, and P_i would need to point to O_{i-1} (or to M, if $i = 1$).

However, now observe that, in order to convert a $(2,\beta)$-ruling set into a solution of this encoded version, it would take β rounds of communication, because nodes not in the set need to figure out their distance from the set. We need to be very careful about this when proving lower bounds: suppose we use round elimination to prove an $f(\beta)$ lower bound for the version of the problem encoded in the node-edge formalism, for some function f. This would not give an $f(\beta)$ lower bound for $(2,\beta)$-ruling sets, but only an $f(\beta) - \beta$ lower bound. Depending on f, this could matter a lot.

5.8 Problems vs Family of Problems

Observe that all the considered examples were problems defined on 3-regular graphs. In fact, we used sentences like "let's see how to define, in the node-edge formalism, the problem of computing an MIS on 3-regular graphs". Observe that we never defined "MIS" or "Maximal Matching", we always only defined them on graphs of a fixed degree. In fact, we could define (by considering a suitable generalization of the node-edge formalism) "MIS on graphs of degree ≤ 100", but we

cannot define MIS in the node-edge formalism. The reason is that there are infinite values of Δ, but we want our problems to have finite description (so that we can apply round elimination on them).

The workaround is the following. We don't consider MIS to be a problem, but we consider it to be a *family of problems*. In other words, we define MIS (on regular graphs) as the family of problems, containing, for each $\Delta \in \mathbb{N}^+$, the following problem:

M^{Δ}

$\mathsf{P}\ \mathsf{O}^{\Delta-1}$

$\mathsf{M}\ \mathsf{OP}$

$\mathsf{O}\ \mathsf{O}$

Yes, we just replaced Δ with a 3. However, think about defining, e.g., Δ-coloring. Then it is not as easy as making some exponents depend on Δ. We would now have that the number of configurations itself depends on Δ, so we would need a new language for describing our problems. This makes it hard to apply the round elimination technique: as we will see, this technique, as is, can be applied on a given problem, but if we want to apply it on an entire family of problems at once, things get more complicated. However, this also shows how we can step outside the LCLs world:

- According to the notation that we just introduced, MIS is definitely *not* an LCL. It is an infinite family of problems, one for each maximum degree Δ.
- We use round elimination to prove, for each problem in the family, a lower bound.
- We see how the asymptotics goes, and we get lower bounds as a function of Δ.

Chapter 6

Generalizations of the Black-White Formalism

[TODO: two ways of define problems with inputs] [TODO: non-regular graphs]

Chapter 7

Round Elimination

[TODO: define RE, explain techniques, ...]

Chapter 8

Other Lower Bound Techniques

[TODO: Marks, KMW, indistinguishability, ...]

Bibliography

[1] Alkida Balliu, Sebastian Brandt, Yi-Jun Chang, Dennis Olivetti, Mikaël Rabie, and Jukka Suomela. The distributed complexity of locally checkable problems on paths is decidable. In Peter Robinson and Faith Ellen, editors, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, pages 262–271. ACM, 2019.

[2] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. Almost global problems in the LOCAL model. In Ulrich Schmid and Josef Widder, editors, *32nd International Symposium on Distributed Computing, DISC 2018, New Orleans, LA, USA, October 15-19, 2018*, volume 121 of *LIPIcs*, pages 9:1–9:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.

[3] Alkida Balliu, Sebastian Brandt, Dennis Olivetti, and Jukka Suomela. How much does randomness help with locally checkable problems? In Yuval Emek and Christian Cachin, editors, *PODC '20: ACM Symposium on Principles of Distributed Computing, Virtual Event, Italy, August 3-7, 2020*, pages 299–308. ACM, 2020.

[4] Alkida Balliu, Keren Censor-Hillel, Yannic Maus, Dennis Olivetti, and Jukka Suomela. Locally checkable labelings with small messages. In Seth Gilbert, editor, *35th International Symposium on Distributed Computing, DISC 2021, October 4-8, 2021, Freiburg, Germany (Virtual Conference)*, volume 209 of *LIPIcs*, pages 8:1–8:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2021.

[5] Alkida Balliu, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Dennis Olivetti, and Jukka Suomela. New classes of distributed time complexity. In Ilias Diakonikolas, David Kempe, and Monika Henzinger, editors, *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2018, Los Angeles, CA, USA, June 25-29, 2018*, pages 1307–1318. ACM, 2018.

[6] Sebastian Brandt. An automatic speedup theorem for distributed problems. In Peter Robinson and Faith Ellen, editors, *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing, PODC 2019, Toronto, ON, Canada, July 29 - August 2, 2019*, pages 379–388. ACM, 2019.

[7] Sebastian Brandt, Juho Hirvonen, Janne H. Korhonen, Tuomo Lempiäinen, Patric R. J. Östergård, Christopher Purcell, Joel Rybicki, Jukka Suomela, and Przemyslaw Uznanski. LCL problems on grids. In Elad Michael Schiller and Alexander A. Schwarzmann, editors, *Proceedings of the ACM Symposium on Principles of Distributed Computing, PODC 2017, Washington, DC, USA, July 25-27, 2017*, pages 101–110. ACM, 2017.

[8] Yi-Jun Chang. The complexity landscape of distributed locally checkable problems on trees. In Hagit Attiya, editor, *34th International Symposium on Distributed Computing, DISC 2020, October 12-16, 2020, Virtual Conference*, volume 179 of *LIPIcs*, pages 18:1–18:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

[9] Yi-Jun Chang, Tsvi Kopelowitz, and Seth Pettie. An exponential separation between randomized and deterministic complexity in the LOCAL model. In Irit Dinur, editor, *IEEE 57th Annual Symposium on Foundations of Computer Science, FOCS 2016, 9-11 October 2016, Hyatt Regency, New Brunswick, New Jersey, USA*, pages 615–624. IEEE Computer Society, 2016.

[10] Yi-Jun Chang and Seth Pettie. A time hierarchy theorem for the LOCAL model. *SIAM Journal on Computing*, 48(1):33–69, 2019.

[11] Yi-Jun Chang, Jan Studený, and Jukka Suomela. Distributed graph problems through an automata-theoretic lens. In Tomasz Jurdzinski and Stefan Schmid, editors, *Structural Information and Communication Complexity - 28th International Colloquium, SIROCCO 2021, Wrocław, Poland, June 28 - July 1, 2021, Proceedings*, volume 12810 of *Lecture Notes in Computer Science*, pages 31–49. Springer, 2021.

[12] Moni Naor and Larry J. Stockmeyer. What can be computed locally? *SIAM Journal on Computing*, 24(6):1259–1277, 1995.